

Energy management in solar cells powered wireless sensor networks for quality of service optimization

Soledad Escolar · Stefano Chessa · Jesús Carretero

Received: 22 August 2012 / Accepted: 19 February 2013 / Published online: 30 April 2013
© Springer-Verlag London 2013

Abstract Sensor nodes equipped with solar cells and rechargeable batteries are useful in many outdoor, long-lasting applications. In these sensor nodes, the cycles of energy harvesting and battery recharge need to be managed appropriately in order to avoid sensor node unavailability due to energy shortages. To this purpose, we suggest sensor nodes to be programmed with alternative scheduling plans, each corresponding to a given energy requirement and meeting a given quality level. Thus, sensor nodes can select the scheduling plan that best suits to the expected energy production and the residual battery charge, in order to avoid sensor nodes' unavailability. We then propose an algorithm for the selection of the scheduling plan that aims at keeping the overall energy consumption neutral and that can sustain the sensor nodes' activities uninterruptedly.

Keywords Power management · Wireless sensor networks · Quality of service · Solar cells · Adaptive duty cycling

1 Introduction

Wireless sensor networks (WSN) are composed of wireless embedded devices, called sensor nodes, capable to monitor

in real-time different environmental phenomena through transducers and to forward the measured data to a special node called gateway or sink, where data are collected for further processing.

A sensor node is typically powered by limited-capacity batteries. The batteries feed the sensor node circuitry by providing the current draw required to sustain the operation of its components. A sensor node integrates several physical components (e.g. radio, transducers, microcontroller, and flash memory), each of which may operate in different states of energy. The sensor node energy consumption is then computed as the sum of the individual consumptions of each component at each state; hence, the sensor node lifetime is the time taken to discharge its battery below the minimum charge level that is required to sustain the sensor nodes operation.

To provide potentially infinite lifetime, recent studies have investigated the use of energy harvesting to power on the sensor nodes [19]. Unfortunately, in realistic scenarios, energy harvesting is generally intermittent and insufficient to constantly fulfill the needs of the sensor nodes. For this reason, the batteries are still necessary: they are recharged when the energy-harvesting system produces an excess of energy and they sustain the sensor node when the energy production of the energy harvesting is not sufficient. However, it could even happen that the battery level is too low and the energy-harvesting system is not able to produce enough energy to recharge the battery, which results in (temporary) periods of sensor node unavailability. To reduce these periods (and possibly to avoid them), the energy harvesting and battery recharge cycles need to be managed appropriately.

In this paper, we consider WSN for outdoor monitoring applications, where the sensor nodes are exposed to different sources of energy found naturally in the

S. Escolar (✉) · J. Carretero
Department of Computer Science, University Carlos III of Madrid, Madrid, Spain
e-mail: sescolar@arcos.inf.uc3m.es

S. Chessa
Department of Computer Science, University of Pisa, Pisa, Italy

S. Chessa
ISTI-CNR, Pisa, Italy

environment, such as the sun, vibration, or acoustic noise. Under this assumption, this work brings together the research areas of green computing and quality of service (QoS) of WSN applications. To achieve this, we define an energy-harvesting system that exploits the energy obtained from the solar light to sustain the sensor node operation and charge its batteries. The main contributions of this paper are as follows: (1) a model for the energy harvesting by means of solar cells; (2) a model that allows to express the QoS of the WSN applications from the user perspective; and (3) an efficient algorithm that optimizes the QoS of the WSN applications accordingly to the estimated energy production from the energy-harvesting system. The algorithm has a low complexity so that it can be executed on low-power sensor nodes. Its goal is to maintain the level of service required by the user as long as possible while the energy production allows it. On the other hand, when the estimated energy production is insufficient to maintain the sensor node operations at the current level, the system progressively degrades the sensor node operation level up to the idle state, that is to say, no operation.

We stress that our view of QoS in WSNs is rather different than the usual view of QoS in the WSN literature. As the primary goal of a sensor node is to continuously monitor the environment, we observe that this activity can be achieved in different ways corresponding to different quality levels. For example, the sensor node may use different sampling rates where, generally speaking, a higher sampling rate implies higher energy cost and better quality of the monitoring activity, it may monitor the environment by using different sets of transducers with different energy costs and different resolutions, and/or it may use different communication patterns (for example, it may transmit each sample in real time or it may group samples to reduce the communications but affecting the delay with which the user receives the samples). We model the different ways of achieving the monitoring goal as scheduling plans of tasks, where each scheduling plan corresponds to a QoS and to an energy cost, and we propose a model in which a sensor node can select, at discrete times, the scheduling plans to be executed during its lifetime, depending on the residual battery power and the expected energy production.

The rest of this paper is organized as follows: after reviewing the related work in Sect. 2, we present in Sect. 3 the design of our energy-harvesting system based on solar cells. Section 4 provides the energy production model from the solar cell and presents a realistic battery discharge and charge model. Section 5 describes the tasks execution model and battery charge. In Sect. 6, we describe our approach to efficiently adapt the QoS level of the WSN application on the basis of the solar energy to be produced by the solar cell, and in Sect. 7, we provide the simulation results. Finally, Sect. 8 discusses the conclusions and further research.

2 Related work

Green computing is a novel general term referring to the capabilities of the Information Technologies (IT) to minimize their negative impact on the environment, which is generally achieved by reducing the CO_2 emissions in the manufacturing processes as well as their energy consumption [10, 18]. In contrast, since in the WSN field, energy is one of the most limited resources, it must be efficiently managed in each architectural layer involved in the process of WSN application building, from the hardware to the application itself.

Dynamic Power Management (DPM) [3] is a technique aimed at energy saving, which consists in switching off the components of a sensor node when not necessary and in waking them up on demand. The probabilistic energy model proposed in [24] allows to take decisions on when to turn on and off the devices on the basis of event statistics. Sensor node components have different energy states, and the events with a consumption above a certain threshold allow to transit from one state to another. While the model takes into account the energy overhead due to the components' state switching, it does not rely on a realistic battery discharge model as proposed in [22] and further extended in [21]. In particular, in [22], the authors use the Rakhmatov–Vrudhula battery model instead of a linear discharge model (which is the default) and prolong the battery lifetime by scheduling the transitions to a sleep state after packet transmissions and by taking advantage of the recovery battery effect. To the best of our knowledge, neither these works nor other DPM methods focus on the improvement of the QoS of a WSN application while preserving energy or use a realistic charge and discharge model which is derived from an energy-harvesting system.

There exist many works in the literature that deal with QoS in WSN [5, 26]. Most of these works focuses on QoS at network or at MAC layers: they aim at defining routing and MAC protocols able to meet the end-to-end QoS requirements while taking into account energy efficiency. A QoS-aware routing protocol not only has to find a route between a source and a destination but also must find the better route according to a QoS metric. Among these metrics, latency is one of the most used [1, 9, 11]. SPEED [11] and its evolution MMSPEED [9] provide soft real-time delivery guarantee by re-routing packets toward less loaded links. In [1], the problem of finding the optimal path is addressed for WSN real-time applications that use imaging and video sensors. This work aims at finding the lesser delay-constrained path and provides best effort services for throughput. The guarantee of end-to-end bandwidth in wireless ad-hoc networks is the focus of other existing works. In [28], the authors provide an algorithm to distributively compute QoS routes with reserved bandwidth

in TDMA networks. Similarly, in [15], the problem is addressed for wireless multimedia ad-hoc networks. At the MAC layer, a few protocols address the QoS by providing services differentiation based on static or dynamic criteria and by providing a specific-application QoS support. A survey of these protocols is presented in [27].

At the application level, a different approach consists in defining an application-specific QoS metric such as coverage, exposure, deployment, or reliability. In [12], QoS is defined as the optimum number of sensor nodes that should send data. In this sense, the sampling rate may also be considered a QoS metric. There exist practical scenarios in which the power consumption due to the sensing activity could be greater than those due to the communication activity. The survey in [2] discusses the strategies that adapt the sampling rates to the real dynamics of the process in order to make an efficient power management in WSN with energy-hungry sensors, and it defines a taxonomy of adaptive sensing strategies: *hierarchical sensing*, *model-based active sensing*, and *adaptive sampling*. The idea behind the hierarchical sensing is that within a node may coexist several transducers able to monitor the same event, where each one has a different accuracy and, in turn, a different energy consumption. Each strategy must dynamically select the transducer that better performs by trading off energy saving and accuracy. Model-based active sensing consists in predicting the next sample of data given a forecasting model of the sensed phenomenon, which leads to a reduction in the sampling rate. Obviously, the effectiveness of this approach depends on the accuracy of the model.

Adaptive sampling techniques adjust the sensor node sampling rate based on the spatial/temporal correlation of the data and, additionally, on estimations of the energy to be harvested from the environment. Among the forms of energy harvested from the environment, the power produced from the sun can be predicted with reasonable accuracy [4]. This is the case of [13], where a mathematical model is defined to express the energy-neutrality condition, which means that the energy consumed by the sensor node in a period of time is always less or equal than the energy harvested from the environment in the same period of time. Based on this concept, the authors propose an algorithm to find the maximum sampling rate of the sensor nodes that keeps the constraint of energy neutrality. The algorithm proceeds by assigning to each time slot a sampling rate. Another example is [17] that considers alternative tasks with discrete service levels and proposes a dynamic programming algorithm that selects the task to be executed for each time slot in a given time horizon.

Our proposal is framed within adaptive sampling techniques that use an energy-harvesting system to recharge the batteries of the sensor node. In our work, we formalize the

QoS as a measure of quality level of the tasks executed by the sensor nodes, and we propose an algorithm aimed at optimizing the QoS while maintaining the system energy-neutral in each considered period of time. For this purpose, we dynamically schedule the tasks that will be executed in the time, on the basis of the estimated energy production of the solar cell and of the tasks' energy costs and quality level. Differently than the above works (except [13] and [17]), we discretize the time axis into slots and, at the end of each one, we take the decision of re-optimizing the scheduling for all slots in the considered period of time. With respect to [13], we do not only consider the adaptation of the duty cycle, but also consider different scheduling plans that may use different duty cycles, transducers, and/or communication patterns. Differently than [17], we consider the constraint of energy neutrality and we focus on a low complexity algorithm that can be easily embedded in low-power sensor nodes. In a preliminary work [8], we addressed a similar problem which only considers the optimization of the sampling rate.

3 Energy-harvesting system

We have designed an energy-harvesting system that produces energy from the sunlight. The system is designed to be robust, and it includes ultracapacitors and rechargeable batteries that power the sensor node in conditions of darkness or deficient sunlight. We establish three priorities of use: by default, the sensor node is powered directly from the solar cell, second from the ultracapacitors connected in parallel to the cell, and third from the two rechargeable batteries. Based on this design, we present the requirements of the solar cell and the sensor node.

Solar cells are composed of wafers, in the order of micrometers thick, made of some semiconductor material at crystalline state, typically silicon, with the property of converting the energy of the photons in electricity by the photovoltaic effect. The efficiency of a solar cell (η) is a measure of the energy conversion efficiency of the incident light absorbed by a solar cell into electric power, computed as $\eta = \frac{P_{\max}}{D \times S}$, where $P_{\max}$ is the maximum output power provided by the cell (in Watts), D is the irradiance, defined as the density of incident power on a surface under standard assumptions (in W/m^2), and S is the surface area of the solar cell (in m^2). For this work, we selected the MSX-005F solar cell [6] manufactured by BP-SOLAR, a polycrystalline cell that provides a maximum current of 150 mA. Table 1 summarizes the specifications of this cell.

The solar cell efficiency depends on the amount of light that it is able to absorb. In turn, this depends on intrinsic factors of the region where it is located: its geographic

Table 1 Specifications of MSX-005F solar cell

Parameter	Value
Specified load voltage (V_{ld})	3.3 V
Typical current at Vld (I_{ld})	150 mA
Open circuit voltage (V_{oc})	4.6 V
Short circuit current (I_{sc})	160 mA
Maximum output power (P_{max})	500 mW
Surface area (S)	36 cm ²

coordinates, the solar time, its inclination with respect to the sun rays (which should be close to 90 degrees to obtain the maximum efficiency), etc. Consider, for example, the sun irradiance in Spain, where the system was deployed for some tests. For each Spanish province, given its latitude and longitude, we obtained the average irradiance in each month of the year from RetScreen NASA program [16]. To ensure that the system works in any location, we took the lowest irradiance, which corresponds to the Lugo province (in the North West) during December. Hence, in this case, the efficiency of the solar cell is $\eta = 11.38 \%$.

To validate our system, we configure our simulation by considering the energy consumption parameters of MicaZ [7], a well-known platform for sensor networks applications. Note, however, that any other platform with similar energy requirements can be used instead. MicaZ is composed of a 7 Mhz μ controller, a Zigbee-compliant CC2420 radio, 3 LEDs, and an external flash memory. To provide capacity of sensing, it may be integrated with different sensor boards. MicaZ is powered by 2-AA batteries, which should provide a voltage between 2.7 and 3.3 for the correct working of the sensor node. According to the documentation, the energy requirements of MicaZ are shown in Table 2. The power demand of MicaZ depends on the

Table 2 MicaZ energy requirements

Parameter		Value		
External power		2.7–3.3V		
μ controller	Active		Idle	
	8 mA		<15 μ A	
Radio	Tx	Rx	Idle	Sleep
	17.4 mA	19.7 mA	20 μ A	1 μ A
Flash	Standby	Read	Write	Load
	2 μ A	4 mA	15 mA	2 μ A
LEDs		On		Off
		22 mA		0.0 mA

components that need to be on in the application and in which mode. In the worst case, if we take into account the maximum current draw of each component (whose sum is 64.7 mA according to Table 2), the power required by MicaZ is $64.7 \times 3.3 = 213.5$ mW. However, it is typically sufficient to guarantee 100 mW to power the device.

Knowing the energy requirements of all components and the I_{ld} provided by the cell, we estimate that nearly 100 mA are needed to charge the batteries of the sensor node. With this current and assuming an initial capacity of the batteries of 2,500 mAh, the system takes about 25 h to charge the batteries. The discharge time in the worst case is $\frac{2,500 \times 1.5 \times 2}{64.7 \times 3.3} = 35.12$ h.

4 Energy production model

We present in this section a realistic battery charge and discharge model which is derived from the energy-harvesting system presented in Sect. 3. For simplicity, we assume that the solar cell is used to directly charge the batteries of the sensor node. Thus, we need to formulate first the energy production function from the solar cell and second the charge and discharge model based on the sensor node energy requirements.

To compute the energy production of the solar cell, we take into account the relevant parameters of the solar cell (η , S) and the average monthly solar irradiance (D) in some location. Thus, the output power is computed as $P_{out} = \eta \times D \times S$ and the average energy production E_p of the solar cell is P_{out}/V_{ld} . Figure 1 shows the energy production by taking the values of average monthly irradiance in Madrid.

However, to obtain the instantaneous irradiance of the solar cell at time t , we should apply a correction factor

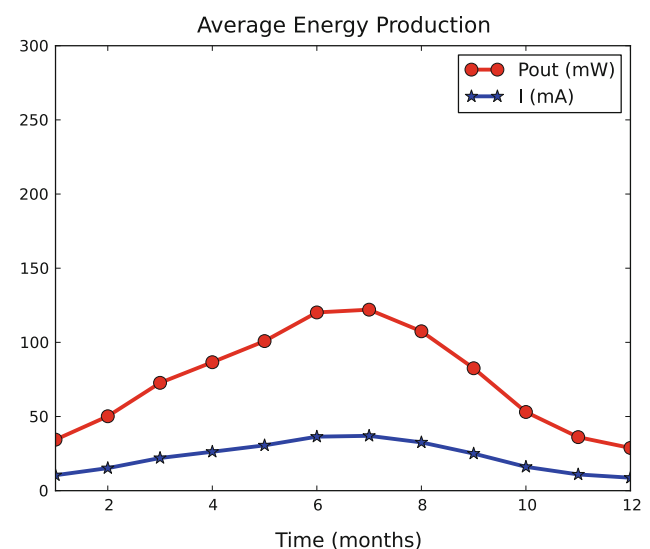


Fig. 1 Average monthly output power and energy production taking the average monthly solar irradiance in Madrid

depending on the current day and solar time. For example, in hours with no solar light, the energy production is close to zero. Thus, at time t , the instantaneous irradiance can be expressed as $D(t) = \frac{(t-h)^2}{4p} + D_{\text{month}}$, where t is any hour between 0:00 and 23:00 **h is the hour of maximum solar light (12:00 am)**, D_{month} is equal to the average irradiance in a month and $p = \text{STDHOURS}_{\text{month}}/D_{\text{month}}$, where $\text{STDHOURS}_{\text{month}}$ is the number of standard hours of solar light in a month. As shown, the irradiance at time t is a quadratic function which draws a parabola with vertex at 12:00 in the noon. By taking different values of irradiance, we can obtain the instantaneous irradiance in any hour of the day, as shown in Fig. 2. Then, the estimated energy production at time t is $E_p(t) = \frac{D(t) \times \eta \times S}{V_{\text{ld}}}$. In Fig. 2, the point (around 6:00) in which energy production begins

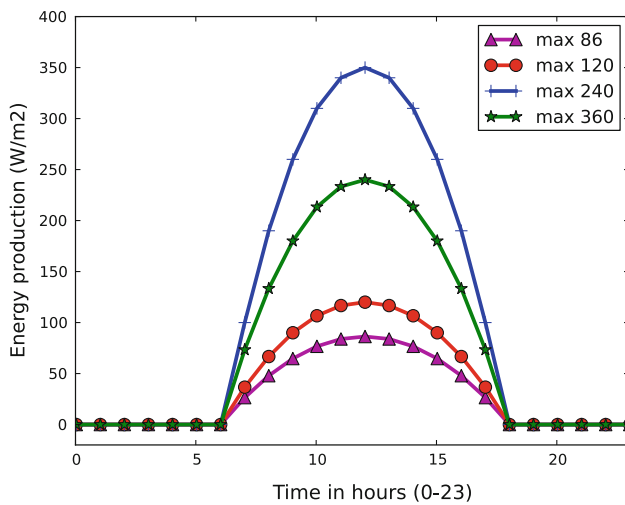
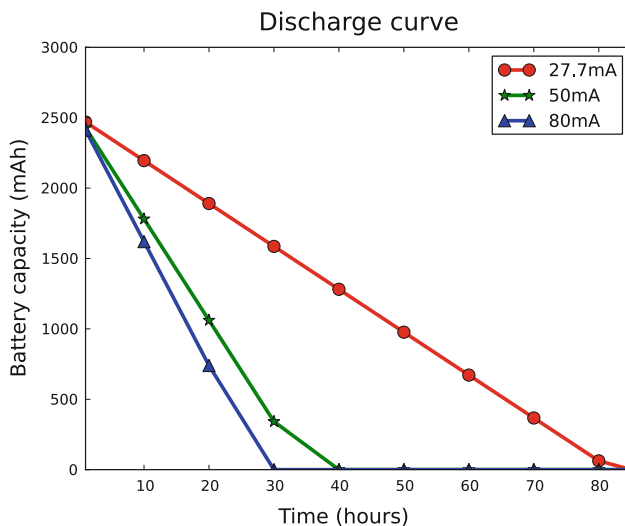


Fig. 2 Irradiance per hour for different D_{month}



corresponds to the sunrise and the point (around 18:00) in which energy production ends corresponds to the sunset.

4.1 Battery discharge and recharge model

The battery discharge model depends mainly on the power requirements of the node, in our case MicaZ. As explained, MicaZ is powered by two batteries of 1.5 V with capacity ranging between 1,500 and 3,000 mAh. To illustrate how these batteries are drained, we consider three different sample applications corresponding to three different operation levels in the sensor node, which have average current draws of 27.7, 50, and 80 mA, respectively. Figure 3 on the left illustrates the discharge curves for every current draw and a battery capacity of 2,500 mAh.

In the system proposed, where the batteries are charged from the solar cell, they experiment an increase in the stored energy according to the energy production. Figure 3 on the right shows the time of charge (in hours) required to completely charge a battery with capacities of 1,500 and 2,500 mAh assuming the energy production obtained from the average monthly irradiance in Madrid (see Fig. 1).

5 System model

We consider a WSN for outdoor monitoring applications, where the sensor nodes are powered by batteries that are recharged by solar cells. Nodes drain the energy from their batteries accordingly to the application they are executing. The application defines an energy consumption pattern, which is characterized by the time of usage of its hardware components in the different operational states and by the energy consumption (or current draw) that each component consumes in each state.

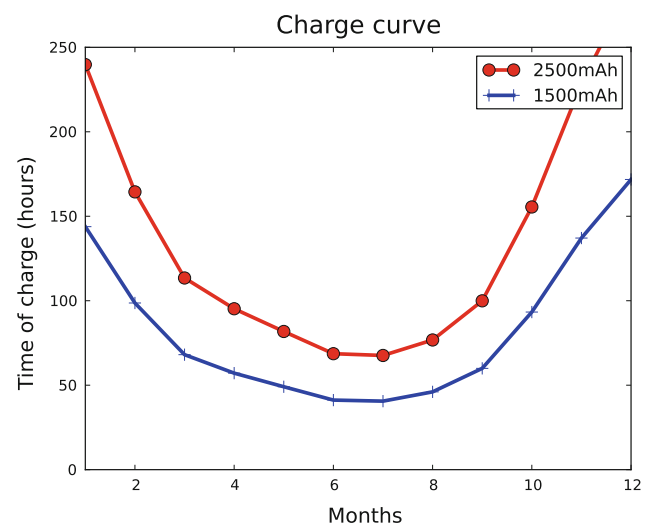


Fig. 3 Battery discharge (on the left side) and charge curves (on the right side)

5.1 Task execution model

A sensor node is modeled as an automation that executes predefined tasks. In a given period of time, the sensor node may be assigned to execute a subset of the available tasks $\Theta = \{T_0, \dots, T_n\}$. A task T_i with $i > 0$ is represented by a triple $T_i = \langle t_i, p_i, w_i \rangle$, where t_i is the time of execution of the task, p_i is the period of execution of the task (note that must be $p_i > t_i$, otherwise the period of execution of the task cannot be met), and w_i is the energy consumption of the node due to the execution of task T_i . The special task T_0 (also called Idle task) is executed when there is no other task to execute. For this reason, T_0 does not have a period and a predefined execution time, and its energy consumption w_0 is defined as the energy consumed per unit of time. In many cases during the execution of the idle task, the processor and most hardware components are switched in a low consumption state in order to save energy. Hereafter, we denote the execution of T_0 for a duration t as T_0^t . The tasks are defined during the design of the sensor node application, in such a way that they are schedulable, that is, there exists a scheduling plan for all these tasks such that they can be executed according to their period and with a bounded jitter. Note that, in order to save the battery capacity and to better exploit the harvesting of energy, it is typical in sensor networks that all the tasks in a scheduling plan of a sensor node are executed in a small fraction of the time, in order to let the sensor spend most of the time executing the idle task to save the battery charge and possibly to recharge it by means of the harvested energy. In the rest of this paper, we will disregard all aspects related to the scheduling of the tasks and we refer to [14, 20, 23, 25] the readers interested in this topic.

A scheduling plan P that executes all the tasks in Θ can be represented as a finite sequence of tasks. Note that since the tasks are periodic, the scheduling plan P is cyclic and the total execution time of the cycle of P equals to the Least Common Multiple (LCM) of the period of the tasks, i.e. $t_P = LCM(T_0, \dots, T_n)$. For example, in a simple case in which $\Theta = \{T_0, T_1\}$ and $T_1 = \langle 1, 3, w_1 \rangle$, a possible scheduling plan for these tasks is $P = \langle T_1, T_0^2 \rangle$. The cost of execution of the scheduling plan P per unit of time is then given by the following:

$$c = \frac{1}{t_P} \left(\sum_{T_i \in \Theta, T_i \neq T_0} \left(w_i \times \frac{t_P}{p_i} \right) \right) + \frac{1}{t_P} \left(t_P - \sum_{T_i \in \Theta, T_i \neq T_0} \left(t_i \times \frac{t_P}{p_i} \right) \right) \times w_0.$$

where the first sum accounts for the cost of the tasks that are not idle, each of which may be executed more than once in P , and the second term accounts for the cost of the

idle task, that is executed in the time remaining after the execution of the other tasks. With simple algebra, Eq. 1 can be rewritten as follows:

$$c = \sum_{\substack{T_i \in \Theta \\ T_i \neq T_0}} \frac{w_i}{p_i} + w_0 \left(1 - \sum_{\substack{T_i \in \Theta \\ T_i \neq T_0}} \frac{t_i}{p_i} \right). \quad (1)$$

5.2 Scheduling plan example

Consider three sets of tasks Θ_1, Θ_2 , and Θ_3 composed as follows: $\Theta_1 = \{T_{1,0}, T_{1,1}, T_{1,2}\}$, $\Theta_2 = \{T_{2,0}, T_{2,1}, T_{2,2}\}$, and $\Theta_3 = \{T_{3,0}, T_{3,1}, T_{3,2}\}$. In the definition of the tasks, the first subindex corresponds to the set of tasks and the second subindex corresponds to the task. The Idle task is denoted as $T_{1,0}$, $T_{2,0}$, and $T_{3,0}$ in Θ_1 , Θ_2 , and Θ_3 , respectively, and, as explained, they do not have a predefined execution time and period. The rest of the tasks are defined as follows: $T_{1,1} = \langle 2, 10, 5 \rangle$, $T_{1,2} = \langle 2, 10, 8 \rangle$; $T_{2,1} = \langle 3, 15, 3 \rangle$ and $T_{2,2} = \langle 8, 15, 6 \rangle$; and $T_{3,1} = \langle 2, 20, 3 \rangle$ and $T_{3,2} = \langle 3, 20, 4 \rangle$.

The description of the tasks in Θ_1 is as follows: $T_{1,1}$ samples an energy-hungry transducer of temperature and $T_{1,2}$ turns the radio on and, after receiving the messages directed to itself, sends all the messages (including the data read from the transducer) to the next hop and turns the radio off; in Θ_2 , the task $T_{2,1}$ samples a temperature transducer and $T_{2,2}$ turns the radio on, receives the messages from its neighbors, computes the average of the temperature data, sends a single message including the result and, finally, turns the radio off; And finally, in Θ_3 , the task $T_{3,1}$ samples a temperature transducer and $T_{3,2}$ turns the radio on to send a single message with the temperature data and turns the radio off.

Let us consider a scheduling plan for each set of tasks: $P_1 = \langle T_{1,1}, T_{1,2}, T_{1,0}^6 \rangle$, $P_2 = \langle T_{2,1}, T_{2,2}, T_{2,0}^4 \rangle$, and $P_3 = \langle T_{3,1}, T_{3,2}, T_{3,0}^{15} \rangle$. When a plan is selected to be executed, all of its tasks are cyclically executed, such as Fig. 4 shows. Knowing the cost of each task, we can compute the cost of the scheduling plans as indicated in Eq. 1.

An initial assignment of scheduling plans P_1 , P_2 , and P_3 to the k slots should be done in such a way that the assignment maximizes the overall QoS while the system is kept energy-neutral. Figure 5 above shows an example of initial assignment of the scheduling plans to the k slots. The assignment matches the most consuming plans to the slots of time that predictably coincide with the hours of more solar light and the lowest consuming plans to the slots in the night and hours of less solar light. However, it is possible that the energy harvested from the system is higher (or lower) than the predicted for each slot, and then, a new assignment should be computed to adapt the solar conditions. We call re-optimization to this process. This fact is shown in Fig. 5 below, where at approximately

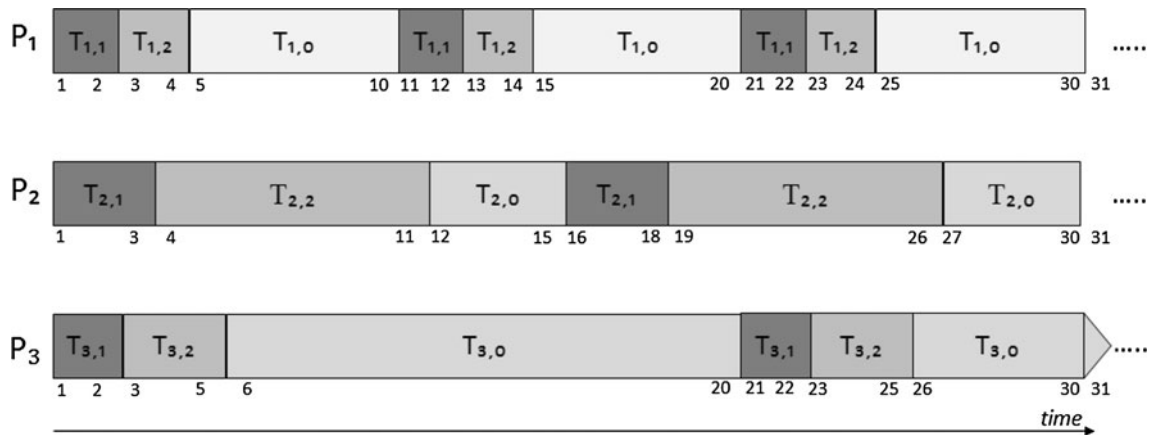


Fig. 4 An example of execution of the scheduling plans P_1 , P_2 , and P_3

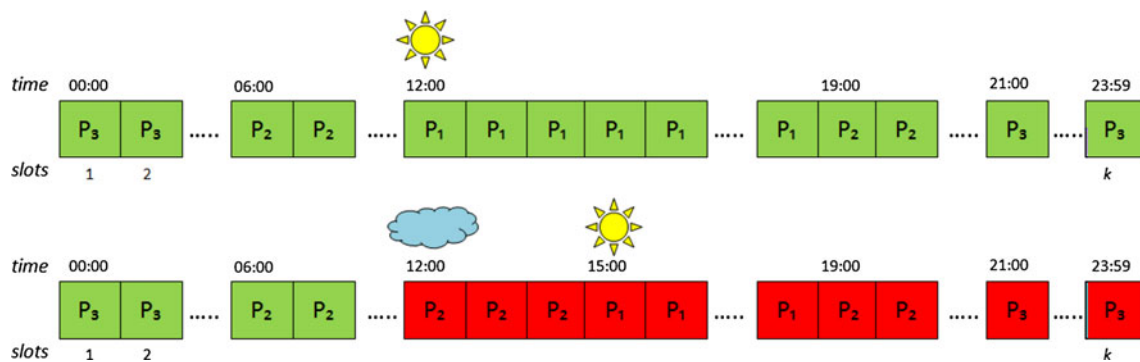


Fig. 5 An example of initial assignment (*above*) and re-optimization (*below*) of the assignment

noon, the solar conditions change with regard to the expected and a re-assignment of scheduling plans to the following slots (marked in red) must be performed.

5.3 Energy model

Let $B_{\max}$ be the maximum battery capacity and $B_{\min}$ be the minimum battery capacity necessary to maintain the sensor node working.

Consider a sensor node connected to our solar energy-harvesting system as explained in Sect. 3. The solar cell obtains the energy from the sun to charge the node batteries according to the light conditions at each moment. Thus, the energy available in the batteries may increase (or decrease) depending on the solar irradiance received in a place and instant of time. Here, the system could be compared to a producer-consumer system, where the producer is the solar cell and the consumer is the sensor node.

Let $e_p(t)$ be the energy produced by the solar cell at time t and $e_c(t)$ be the energy consumed by the sensor node at time t , and B_t be the battery level at time t . Let also $\phi \in$

$[0, 1)$ be the charging efficiency of the battery and the rectifier function $[x]^+$ be defined as follows:

$$[x]^+ = \begin{cases} x & x \geq 0 \\ 0 & x < 0 \end{cases} \quad (2)$$

According to [13], and disregarding the power leakage of the battery, energy production and consumption at time t are subject to the constraint:

$$B_0 + \phi \int_0^t [e_p(t) - e_c(t)]^+ dt - \int_0^t [e_c(t) - e_p(t)]^+ dt \geq 0 \forall t \in [0, \infty) \quad (3)$$

where the second term accounts for the energy which is produced and not consumed (which recharges the battery), and the last term accounts for the energy which is drained from the battery. This equation considers the fact that the battery is not an ideal energy buffer, and it assumes that the excess energy that is neither used nor accumulated in the battery is dissipated by the system. Letting B_t be the battery level at time t , we thus have that $0 \leq B_t \leq B_{\max} \forall t \in [0, \infty)$.

As observed in [13], an energy-harvesting sensor node can operate uninterruptedly if its energy management is *energy-neutral*. Considering a reference period of time $[t', t'']$, the energy management of a sensor node is energy-neutral if $B_{t'} = B_{t''}$. In our case, considering that the energy-harvesting system is based on solar cells, the energy production has a cycle of 24 h, for this reason, we aimed at devising an energy-management strategy that is energy-neutral in the 24 h.

6 Energy-management optimization

As shown in Sect. 4, the energy production given by a solar cell is predictable, although it is uncontrollable. We exploit the predictability of the energy production in order to devise energy-neutral strategy for the energy management of the nodes. We assume that alternative scheduling plans are available and that these scheduling plans are all energy-neutral, i.e. sustainable, at least under some environmental conditions. For example, a scheduling plan may not be sustainable for winter cloudy days, but it may be sustainable in a summer sunny day. Two scheduling plans may implement the same functionalities with different duty cycles, or it may implement different functionalities, as defined by the user.

Although, in principle, knowing all the parameters of the energy production, it could be possible to select the best scheduling plan that is always sustainable, we observe that these parameters may not be known in advance and they may also change with the time. Thus, the energy-management strategy should be adaptable to different environmental conditions, and hence, it should switch among the scheduling plans depending on such conditions.

Let us assume there are n possible sets of tasks to be executed $\Theta_1, \dots, \Theta_n$. For each subset Θ_j , a scheduling plan P_j is defined with an energy cost per unit of time c_j and a user defined quality level q_j . The main objective of our energy-management algorithm is to select the scheduling plans in order to maximize the overall quality level. To this purpose, we consider a time frame of 24 h, and we discretize the time axis into k slots of duration $\Delta T = \frac{3,600 \times 24}{k}$ seconds. For each slot i the energy-management algorithm selects the scheduling plan $S[i]$ (thus $S[i] = P_j$ for some j), corresponding to quality level $q(i) = q_j$ and energy consumption $c(i) = c_j$. Let also $e_p(i)$ be the energy production during slot i and $e_e(i)$ be the expected energy production in slot i , which can be estimated by the energy production model discussed in Sect. 4.

We denote with $B(i)$ the battery level at the end of slot i ; if the duration of the slots is small enough, we can assume that if the charge tends to overflow the battery above $B_{\max}$, then the battery charge will be $B_{\max}$ also at the end of the slot.

Similarly, if the charge tends to underflow the battery below 0, then the battery charge will be 0 also at the end of the slot. This assumption is motivated by the fact that the slots are short, the energy production will keep the same trend while the energy consumption will remain almost the same. Under this assumption, we can express the estimated battery level $B(i)$ for all $i \in [1, k]$ as follows:

$$B(i) = \max\{B_{\max}, B(i-1) + \phi[e_e(i) - c(i)]^+ - [c(i) - e_e(i)]^+\} \quad (4)$$

where $B(0)$ is the initial battery level at the beginning of slot 1.

The quality of service (QoS) of the assignment is defined as the sum of the quality levels in all the slots, and the problem of assignment can be formalized as an optimization problem as follows:

$$\begin{aligned} \max \quad & \sum_{i=1}^k q(i) \\ B(0) \leq & B(k) \\ B_{\min} \leq & B(i) \forall i \in [1, k] \end{aligned} \quad (5)$$

The first constraint of the above optimization problem states that the assignment of the scheduling plans made by the algorithm is energy-neutral. The second constraint states that the battery level can never be below $B_{\min}$ because otherwise the sensor node would have to stop its operations. Note also that the expression of $B(i)$ is not linear, thus leading to a nonlinear optimization problem.

To simplify the problem, we can consider the case of an ideal battery, in which the initial charge is high enough and the maximum charge is infinite (this assumption can be considered realistic also if the battery has a very high capacity). In this case, we can disregard the second constraint, and we can rewrite the optimization problem as follows:

$$\begin{aligned} \max \quad & \sum_{i=1}^k q(i) \\ \sum_{i=1}^k e_e(i) \geq & \sum_{i=1}^k c(i) \end{aligned} \quad (6)$$

where the constraint states that the energy production during the 24 h should exceed the energy consumption.

We observe that this problem differs from the Unbounded Knapsack Problem (UKP) since in Eq. 6, there is a constraint on the number of items (scheduling plans) that can be taken (there are exactly k slots), while in UKP, there is not such a limit. However, UKP can be reduced to 6 as follows. Let us consider an instance of UKP:

$$\max \sum_{i=1}^n x_i \times q_i, \quad \text{where} \quad \sum_{i=1}^n x_i \times c_i \leq W \quad (7)$$

where x_i is a positive integer and represents the number of times an item P_i with cost c_i and value q_i is selected. W.l.g. let us assume that $c_1 \leq c_2 \leq \dots \leq c_n$. It is immediate that, for each admissible solution of the problem, must be $\sum_{i=1}^n x_i \leq W/c_1$, which means that the number of items that can be selected in an admissible solution is at most $m = W/c_1$. We can then transform this problem in a set of m instances of problem 6, where k varies between 1 and m . In particular, for each $i \in [1, n]$, let $q(i) = q_i, c(i) = c_i, \sum_{i=1}^k e_e(i) = W$, and consider all the resulting instances of problem 6:

$$\forall k \in [1, m] \max Q_k = \sum_{i=1}^k q(i), \quad \text{where } \sum_{i=1}^k c(i) \leq W$$

By definition of problem 6, the result of instance k maximizes the value of the selected items in the case where exactly k items can be selected. Hence, the solution to the instance k' with maximum $Q_{k'}$ is also optimal for problem 7. Consequently, solving problem 6 is at least as difficult as solving UKP, which is known to be NP-Complete. For this reason, we present in the next section a greedy heuristic aimed to solve the problem.

6.1 Initial assignment algorithm

We assume that the scheduling plans $P_1, \dots, P_n$ are preliminarily sorted according to $\frac{q_i}{c_i}$ i.e. $\frac{q_i}{c_i} \geq \frac{q_j}{c_j}$ iff $i < j$ and $q_i > q_j$ if $\frac{q_i}{c_i} = \frac{q_j}{c_j}$. It should be noted that a plan P_j for which $q_j < q_i$ and $c_j \geq c_i$ for some i is inefficient, and it can be excluded a priori.

Let us consider the array $P[]$ that contains the scheduling plans (where, initially, $P[i] = P_i$) and let $q_P(i)$ and $c_P(i)$ be the quality level and the energy cost of the scheduling plan contained in $P[i]$, respectively. Let also assume that a time frame of 24 h is divided into k slots, where r_1 is the slot immediately after the sunrise, and r_2 is the slot immediately after the sunset. Let $e_p(s)$ be the energy production in the slot s and $e_e(s)$ be the estimated energy production in the same slot. Let $S[]$ be the vector containing the assignment of the scheduling plans to the slots. Then, for a given assignment $S[]$, $c(s)$ is the energy consumption in slot s , and the expected energy budget of $S[]$ is given by $B(k) - B(0)$.

We denote with $P[]|_q$ the vector obtained from $P[]$ by removing all elements that have quality level smaller or equal to q (i.e. $q_P(i) \leq q$). Similarly, we denote with $P[]|^c$ the vector obtained from $P[]$ by removing all elements that have a cost larger or equal to c (i.e. $c_P(i) \geq c$). The operations $P[]|_q$ and $P[]|^c$ return an empty vector, denoted with $\emptyset$, if no elements in $P[]$ satisfy the requested property. Note that, if $P[]$ is sorted, the vectors $P[]|_q$ and $P[]|^c$ are also

sorted according to the same order, and both operations can be performed in a linear time with the size of the vector.

The pseudo-code for the initial assignment is presented in Algorithm 1. This algorithm progressively achieves a (sub)optimal solution in several iterations. It starts initializing the most efficient plan $P[1]$ to all of the slots. After assigning a plan, the algorithm evaluates the energy-neutral condition by comparing the battery level at slot 0 and at slot k . An optimal solution is achieved if the battery level is equal at these two slots and the battery level is at least B_{min} in all slots. Otherwise, two cases may occur: (1) If the battery level at slot k is larger than the battery level at slot 0 ($B(k) - B(0) > 0$) and $B(i) \geq B_{min} \forall i$, this means that the estimated energy production is higher than the consumed energy, and therefore, there is a choice of finding a better assignment; then, the algorithm selects the plan in $P[]$ with the next higher quality (and higher consumption), which replaces the plan assigned in all of the slots in $S[]$ (this replacement continue until the solution remains energy-neutral, and it is performed by the function *Upgrade*); (2) If the battery level at slot k is lower than the battery level at slot 0 ($B(k) - B(0) < 0$) or there exists a slot i for which $B(i) < B_{min}$, this means that the current assignment is not sustainable, and then, the algorithm must select the plan in $P[]$ with the next lower cost (and lower consumption), which replaces the current plan in all of the slots (this replacement stops as soon as the solution becomes energy-neutral, and it is performed by the function *Downgrade*). The algorithm iterates until there exists no plan in $P[]$ that improves the achieved solution.

Algorithm 1 Initial Assignment Algorithm

Require: $P[]$: the vector of scheduling plans.
Ensure: $S[]$: the scheduling plans assignment vector.
 $S[s] = P[1] \forall s$
case 1: $(B(k) \geq B(0)) \wedge (B(i) \geq B_{min} \forall i)$
loop
 $P'[] = P[]|_{q_{P(1)}}$
if $B(k) == B(0) \vee P'[] == \emptyset$ **then**
 the solution is optimal; exit;
 $S[] = Upgrade(S[], P'[1])$
 $P[] = P'[]$
end loop
case 2: $(B(k) < B(0)) \vee (B(i) < B_{min} \text{ for some } i)$
loop
 $P'[] = P[]|^c$
if $P'[] == \emptyset$ **then**
 there is not admissible solution; exit;
 $S[] = Downgrade(S[], P'[1])$
 $P[] = P'[]$
if $B(k) \geq B(0) \wedge B(i) \geq B_{min} \forall i$ **then**
 go to case 1;
end loop

The functions *Upgrade* and *Downgrade* are shown in algorithms 2 and 3, respectively. They take a scheduling plan assignment $S[]$ and replace the elements of $S[]$ with a new scheduling plan P_i , starting from an initial slot.

Function Upgrade replaces elements from $S[]$ with scheduling plans that have a better quality level (and, typically, a higher energy costs). For this reason, it starts the replacement from the slots after the sunrise (i.e. at slot r_1), since in these slots, the energy production will compensate for the additional energy costs. The Function Upgrade replaces elements as long as the resulting assignment is energy-neutral, and the battery level is at least $B_{\min}$ for all slots. On the other hand, Function Downgrade replaces elements from $S[]$ with scheduling plans that have a lower cost (and, typically, a lower quality level). For this reason, it starts the replacement from the slots after the sunset (i.e. at slot r_2) in the slots where there is no energy production. The Function Downgrade stops replacing elements as soon as the assignment becomes energy-neutral and the battery level is at least $B_{\min}$ for all slots.

It can be observed that the complexity of Algorithm 1 is $O(n \cdot (k + n))$. In fact, in each iteration of the loop, a scheduling plan P_i is selected and assigned to each one of the k slots by the Function Upgrade (or Downgrade). These functions take complexity $O(k)$. When a plan P_i is selected, it is also removed from $P[]$ (along with all the scheduling plans with a lower quality level or higher cost, depending on the case), thus reducing the number of scheduling plans in $P[]$ of at least 1 element. As a consequence, the main loop is repeated at most n times. Note that the removal of elements from $P[]$ can be done in $O(n)$. Consequently, the overall complexity is $O(n \cdot (k + n))$. Considering that, typically $k \gg n$ holds, the complexity is $O(n \cdot k)$.

6.2 Re-optimization algorithm

Algorithm 1 is used to compute an initial assignment vector that should hold for all the slots in a time frame, based on the estimation of the energy production and the cost of the scheduling plans. Typically, the time frame is 24 h and the Algorithm 1 is thus executed once per day. However, at execution time, the solar conditions may change leading to a production higher or lower than the expected one, making inefficient or non-energy-neutral the initial assignment, which should be adapted to the new conditions. To this end, we propose a re-optimization algorithm, whose pseudo-code is presented in Algorithm 4. This algorithm re-optimizes the assignment starting from the slot x to slot k (assuming that at slot $x - 1$ a difference between the real energy production and the expected one is detected). Two cases may occur: (1) If the expected battery charge at the end of time frame B_k exceeds the initial battery level B_0 for more than a parameter ϵ , the current assignment can be improved to maximize the overall QoS. To this purpose, the algorithm keeps improving the plans in the slots $[x, k]$. This improvement continues until there exists no plan that improves the achieved solution or the assignment becomes non-energy-neutral; (2) Otherwise, if the expected battery charge at the end of time frame B_k is below the initial battery level B_0 for more than a parameter ϵ or $B(i) < B_{\min}$ for some slot i , the algorithm looks for an assignment with a lower cost. To this purpose, the algorithm keeps replacing the plans in slots $[x, k]$ with plans with smaller energy cost.

Algorithm 2 Upgrade Function

Require: $S[]$: the scheduling plans assignment vector; P_i : a scheduling plan.
Ensure: $S'[]$: the resulting assignment vector.

```

 $S'[] = S[]$ 
 $s = r_1; j = 1$ 
while  $((B(k) - B(0) \geq c_i - c(s)) \wedge (j \leq k))$  do
   $H = S'[s]; S'[s] = P_i$ 
  if  $B(i) < B_{\min}$  for some  $i$  then
     $S'[s] = H$ 
    return  $S'[]$  % undo the assignment and exit
   $s = (s + 1) \bmod k; j++$ 
end while
return  $S'[]$ 

```

Algorithm 3 Downgrade Function

Require: $S[]$: the scheduling plans assignment vector; P_i : a scheduling plan.
Ensure: $S'[]$: the resulting assignment vector.

```

 $S'[] = S[]$ 
 $s = r_2; j = 1$ 
while  $((B(k) - B(0) < 0) \vee (B(i) < B_{\min}) \text{ for some } i \wedge (j \leq k))$  do
   $S'[s] = P_i$ 
   $s = (s + 1) \bmod k; j++$ 
end while
return  $S'[]$ 

```

This improvement continues until there exists no plan that improves the achieved solution or the assignment becomes again energy-neutral and the battery level is at least $B_{\min}$ for all slots. It is immediate that the computational cost of this algorithm is $O(k \cdot n)$.

$B(300) \geq B(0)$, which means that the system is sustainable and energy-neutral in a day.

We define six sets of tasks $(\Theta_1 \dots \Theta_6)$ and their corresponding scheduling plans $(P_1 \dots P_6)$, whose description is detailed in Table 3. For each plan, the columns from 2 to 5

Algorithm 4 Re-optimization Algorithm

Require: $S[]$: the scheduling plans assignment vector; $P[]$: the vector of scheduling plans;
 x : the initial slot; ϵ : a tolerance parameter.
Ensure: $S'[]$: the resulting assignment vector.
 $S'[] = S[]$
case 1: $(B(k) \geq B(0) + \epsilon) \wedge (B(i) \geq B_{\min} \forall i)$
 $Replaced = True$
while $Replaced$ **do**
 $s = x; Replaced = False$
while $s \leq k$ **do**
Let P_j be the scheduling plan of $S'[s]$
if $(j < n) \wedge (B(k) - B(0) \geq c_{j+1} - c_j)$ **then**
 $S'[s] = P_{j+1}; Replaced = True$
if $(B(i) < B_{\min} \text{ for some } i)$ **then**
 $S'[s] = P_j$
 $Replaced = False$ % undo the assignment
 $s++$
case 2: $(B(k) \leq B(0) - \epsilon) \vee (B(i) < B_{\min} \text{ for some } i)$
 $Replaced = True$
while $Replaced$ **do**
 $s = x; Replaced = False$
while $s \leq k$ **do**
Let P_j be the scheduling plan of $S'[s]$
if $(j > 1) \wedge ((B(k) < B(0)) \vee (B(i) < B_{\min} \text{ for some } i))$ **then**
 $S'[s] = P_{j-1}$
 $Replaced = True$
 $s++$

7 Evaluation

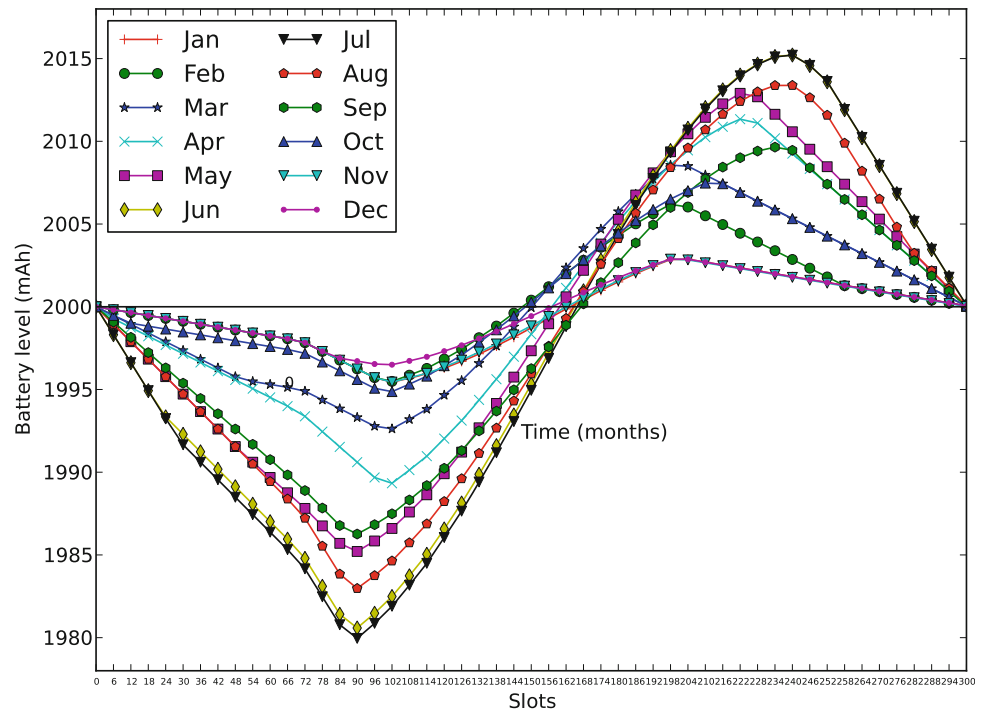
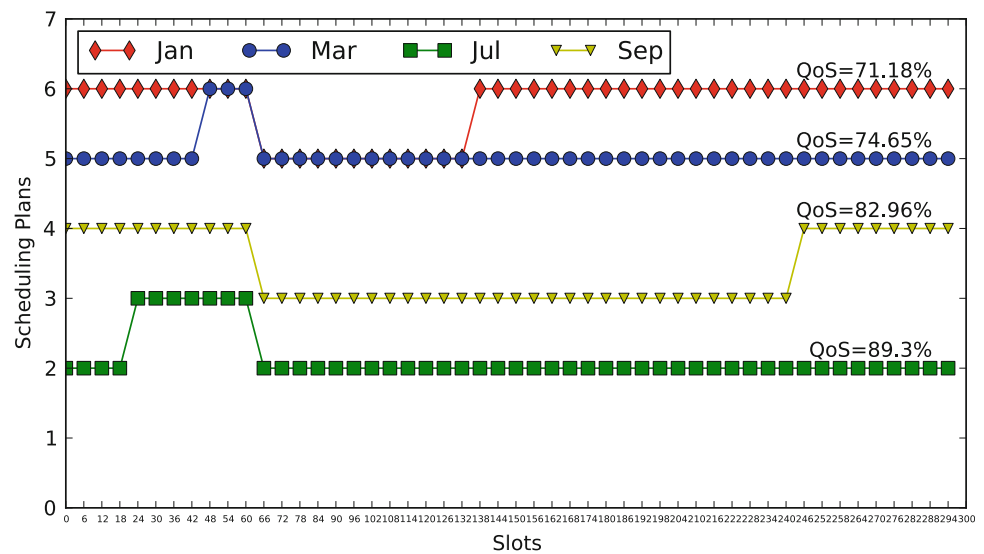
We have evaluated the algorithms proposed by using a simulator implemented in C. We consider a time frame of 24 h composed of 300 slots ($k = 300$) with a duration of 288 seconds each one. To estimate the energy production, we take the average monthly solar irradiance in Madrid (data provided by RetScreen [16]): 84.58, 123.33, 178.75, 212.92, 247.92, 295.42, 300.0, 264.1, 202.9, 130.42, 88.75, 70.83 W/m², corresponding to the months of the year from January to December. We compute the instantaneous energy production at hour t (between 0:00 and 23:00) by using the energy production model described in Sect. 4 and we distribute the energy production $E_p(t)$ equally among each one of the slots in an hour ($\frac{k}{24}$). Additionally, to do more realistic simulations, we varied the first and last light hour accordingly to each month, i.e. the sunrise slot r_1 and sunset slot r_2 . The maximum capacity of the battery $B_{\max}$ was configured to be 2,500 mAh and the minimum $B_{\min}$ to be 1,975 mAh. We fix the battery level at slot 0 to be 2,000 mAh and a tolerance parameter $\epsilon = 10^{-4}$. Thus, to validate the initial assignment algorithm, we need to check

present the definition of the tasks 1–4, respectively (the notation “–” means that the plan has not that task); the column 6 indicates the execution time for the Idle task; and, in subsequent columns, we specify its cost per unit of time c (as defined by Eq. 1), its quality level q (in percentage) and its efficiency, which is computed as $\frac{c}{q}$.

We execute the initial assignment algorithm for the first day of each month of the year, starting at 0:00 am that corresponds to the slot 0. Figure 6 presents the results of its execution. Since at slot 300 the battery level is always larger or equal to 2,000 mAh, the algorithm finds an assignment of scheduling plans to slots such that the system is energy-neutral each month of the year. As shown, there is a clear relationship between the energy production and the battery level: the bigger is the energy production, the larger is the battery level achieved at the last light hour (approximately at slot r_2) and the lower is the battery level before r_1 , due to the fact that the most consuming plans can be assigned when the production is larger. In the figure, the biggest energy production happens in July, where a maximum battery level of approximately 2,015 mAh is achieved at slot r_2 . Correspondingly, the lowest battery level of

Table 3 Parameters of the scheduling plans employed in the simulation

Plan	T_1	T_2	T_3	T_4	T_0	c	q (%)	Efficiency
P_1	$\langle 2, 5, 10 \rangle$	$\langle 1, 5, 15 \rangle$	$\langle 1, 5, 8 \rangle$	–	T_0^1	7.0	100	14.28
P_2	$\langle 1, 10, 3 \rangle$	$\langle 1, 10, 5 \rangle$	$\langle 1, 10, 5 \rangle$	$\langle 1, 10, 7 \rangle$	T_0^6	3.2	90	28.12
P_3	$\langle 5, 15, 10 \rangle$	–	–	–	T_0^{10}	2.0	85	42.50
P_4	$\langle 2, 20, 3 \rangle$	$\langle 4, 20, 4 \rangle$	–	–	T_0^{14}	1.75	80	45.71
P_5	$\langle 20, 30, 10 \rangle$	–	–	–	T_0^{10}	1.0	75	75
P_6	$\langle 26, 30, 2 \rangle$	–	–	–	T_0^4	0.33	70	210

Fig. 6 Battery level during the 1st day of each month of the year**Fig. 7** Initial assignment of scheduling plans to slots for the months of January, March, July, and September

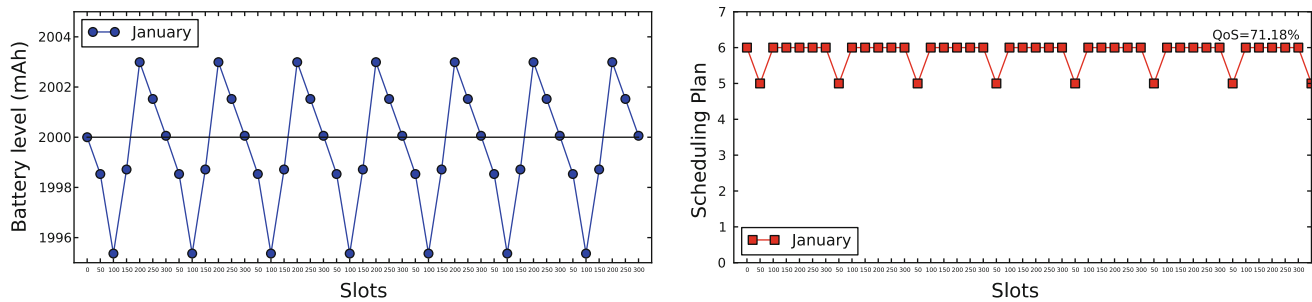


Fig. 8 On the *left side*, the battery level during the first week of January ($k = 2,100$). On the *right side*, the assignment of scheduling plans to the slots of the same week

Table 4 Summary of the QoS obtained for each month of the year after re-optimization due to an overproduction/underproduction of solar energy of 1,5,10, and 20 %

		Jan		Feb		Mar		Apr	
IA		71.18		73.11		74.65		78.88	
O	U	O	U	O	U	O	U	O	U
1	−1	71.2	−	73.2	−	74.7	−	79.2	78.9
5	−5	71.3	−	73.3	−	74.9	−	81.8	78.7
10	−10	71.4	−	73.5	−	75.9	−	82.7	76.5
20	−20	72.2	−	75.3	−	77.1	−	83.5	75.9
		May		Jun		Jul		Aug	
IA		84.63		89.2		89.3		88.5	
O	U	O	U	O	U	O	U	O	U
1	−1	84.7	83.8	89.2	89.1	89.3	89.2	88.5	88.5
5	−5	85.7	81.5	89.4	87.4	89.5	87.5	88.7	87.0
10	−10	87.2	80.0	90.6	86.3	90.7	86.4	89.5	85.7
20	−20	87.7	78.0	94.2	85.7	94.5	85.8	92.7	84.5
		Sep		Oct		Nov		Dec	
IA		82.96		74.0		72.16		70.18	
O	U	O	U	O	U	O	U	O	U
1	−1	83.0	82.4	74.0	−	72.2	−	70.2	−
5	−5	83.7	80.1	74.2	−	72.3	−	70.4	−
10	−10	85.5	77.5	74.5	−	72.4	−	70.4	−
20	−20	86.2	76.8	76.4	−	73.3	−	70.6	−

All data are expressed in percentage

IA initial assignment, O overproduction, U Underproduction, − Non-energy-neutral assignment

approximately 1,980 mAh is achieved also in July, since the most consuming plans are assigned in that month.

Figure 7 shows the result of the initial assignment of scheduling plans to slots and the overall QoS (in percentage). To avoid excessive cluttering, the assignment is only shown for the months of January, March, July, and September. The scheduling plans are assigned according to the

energy consumption and production at each slot. Thus, it is expected that the most consuming plans be assigned in the months of summer and the lowest consuming plans in the months of winter. Once the assignment is done, we compute the overall QoS after the first day of each month as the average of the quality levels corresponding to the scheduling plans assigned. As shown, the maximum QoS (89.3 %) is achieved in July when the production of energy is the biggest and P_2 is assigned to most of the slots (note that P_1 cannot be assigned because it would become non-energy-neutral the system). Only in some slots previous to r_1 (coinciding with the time where there is no energy production), our strategy has to degrade the QoS and select the next lower energy plan P_3 to be assigned. Analogously, the minimum QoS (71.18 %) corresponds to January, where the lowest consuming plan P_6 is assigned to most of the slots except some slots after r_1 (coinciding with the beginning of the energy production). Here, our strategy may upgrade the QoS and select the next higher energy plan P_5 to be assigned.

As the energy consumption is expected to be constant, the battery level draws a pattern that is repeated each day. To show this effect, Fig. 8 on the left side presents the battery level during the first week of January (the worst case). On the right side, we show the initial assignment during that week and the overall QoS.

We also evaluate the re-optimization algorithm. Under an excess (or deficit) of energy production with regard to the estimated production, the re-optimization algorithm is able to adjust the initial assignment of scheduling plans to slots, in such a way that the overall QoS is maximized while the system remains energy-neutral. We consider an overproduction and an underproduction of solar energy of 1, 5, 10, and 20 % with regard to the expected energy production. This overproduction (or underproduction) is applied to all the slots between $[x, r_2]$ (where x is randomly selected, $x \geq r_1$) due to the fact that if we consider that a day is sunnier (or cloudier) than the expected, in general it will be sunny (or cloudy) during most part of the day and not only at a specific slot. We compute the effect of

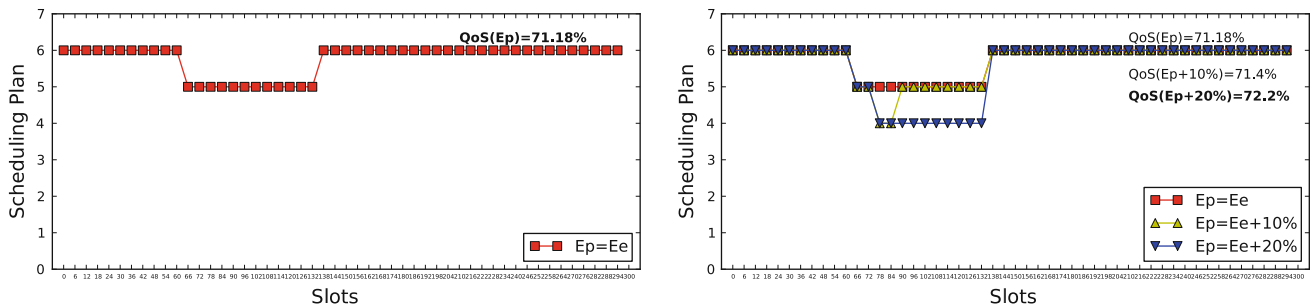


Fig. 9 Re-optimization in January due to a deficit (on the left side) and an excess (on the right side) of solar production (Ep) of 10 and 20 % with regard to the expected production (Ee)

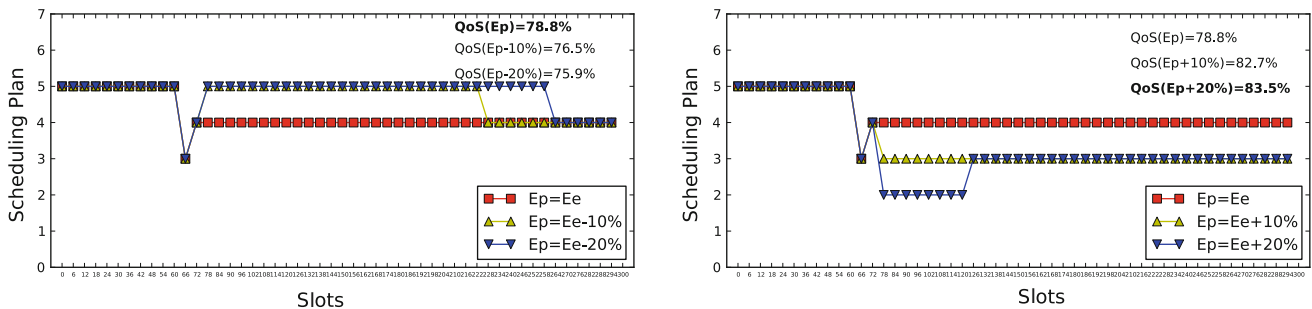


Fig. 10 Re-optimization in April due to a deficit (on the left side) and an excess (on the right side) of solar production (Ep) of 10 and 20 % with regard to the expected production (Ee)

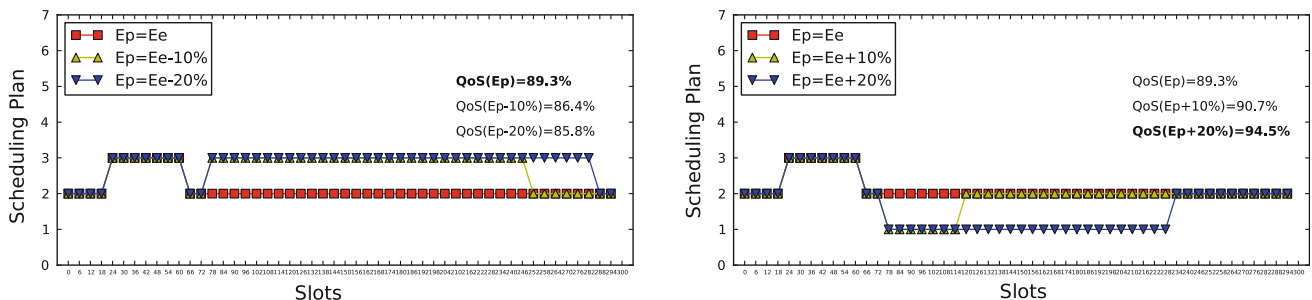


Fig. 11 Re-optimization in July due to a deficit (on the left side) and an excess (on the right side) of solar production (Ep) of 10 and 20 % with regard to the expected production (Ee)

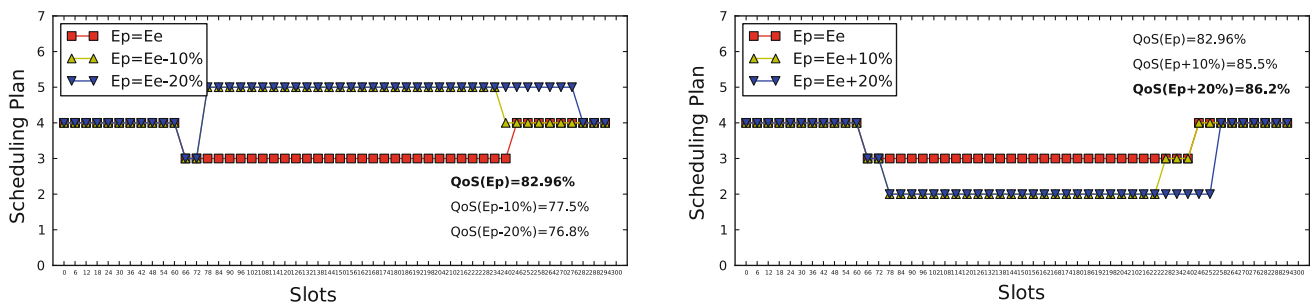


Fig. 12 Re-optimization in September due to a deficit (on the left side) and an excess (on the right side) of solar production (Ep) of 10 and 20 % with regard to the expected production (Ee)

re-optimization on the QoS for all months of the year and present the results in Table 4. For each month, the column on the left presents the QoS when an excess of energy production exists, and the column on the right, the QoS when there is a deficit. A deficit of energy with regard to the estimated energy may make non-energy-neutral the system. This happens in the months where the energy production is low (from October to March), and there exists no a more efficient plan that can be assigned (note that, if in a real application this is an issue, the developer should design the energy-harvesting system and the battery capacity in an appropriate way). In the months with higher production, between April and September, the underproduction results in a new assignment that keeps energy-neutral the system as well as reducing the overall QoS. As observed, the larger is the deficit in the energy production, the larger is the reduction in the resulting QoS, and vice versa. Excess of production of 1 and 5 % do not have significant impact on the overall QoS achieved, which is similar to the QoS achieved by the initial assignment algorithm. An excess of 10 and 20 % increases meaningfully the QoS, especially in the months of summer (from June to September). As an example, in July, we achieve a QoS of 90.7 and 94.5 %, respectively. Figures 9, 10, 11, 12 present the new assignment of scheduling plans to slots after re-optimization in January, April, July, and September as well as the QoS achieved. On the left side of the figures, we present the results when there exists underproduction and, on the right side, when there exists overproduction. A deficit of solar energy implies to assign lower consuming plans which, in turn, reduce the resulting QoS. The worst case happens when the re-optimization algorithm cannot find an energy-neutral assignment (for example, in January). This fact is shown in Fig. 9 on the left, where only the initial assignment is shown. However, as depicted in Figs. 10, 11, 12 on the left, for the other months, the algorithm produces a new valid assignment. Similarly, an excess of solar energy implies to assign higher consuming plans and, in turn, to increase the QoS. On the right, we present the new assignment and its QoS, which improves the QoS achieved by the initial assignment.

8 Conclusions and future work

Energy-harvesting systems (e.g. solar cells) allow to achieve potentially infinite sensor node's lifetime. However, in practice, the production of energy is intermittent and, in some cases, it could be insufficient to constantly satisfy the power demand of WSN applications. In this paper, we model the energy production of the solar cell and the energy consumption of a typical MicaZ sensor node. Assuming that there exist different scheduling plans with

different energy cost and QoS and that we discretize the time axis into k slots, we propose an algorithm to find an assignment of scheduling plans to the slots that aims at maximizing the QoS while keeping the system energy-neutral. Since the energy produced could vary from the expected one, we also propose a re-optimization algorithm that corrects the initial assignment to meet the new energy conditions. We have evaluated the algorithms by simulation and shown how a valid assignment that maintains energy-neutral the system is found for all months of the year. We applied our re-optimization algorithm to show how the assignment is adapted to the conditions of underproduction and overproduction, which is expressed in terms of a lower and higher QoS, respectively. As future work, we plan to dynamically adapt the sampling frequency of the applications according to the estimated production and computing the solar cells requirements based on the power demand of sensor nodes.

Acknowledgments This work has been funded by the Spanish Ministry of Science and Technology under the grant TIN2010-16497 "Técnicas Escalables de E/S en Entornos Distribuidos y de Computación de Altas Prestaciones".

References

1. Akkaya K, Younis M (2005) Energy and qos aware routing in wireless sensor networks. *Cluster Comput* 8:179–188. doi: [10.1007/s10586-005-6183-7](https://doi.org/10.1007/s10586-005-6183-7)
2. Alippi C, Anastasi G, Di Francesco M, Roveri M (2009) Energy management in wireless sensor networks with energy-hungry sensors. *Instrum Meas Mag IEEE* 12(2):16–23. doi: [10.1109/MIM.2009.4811133](https://doi.org/10.1109/MIM.2009.4811133)
3. Benini L, Bogliolo A, De Micheli G (2000) A survey of design techniques for system-level dynamic power management. *IEEE Trans Very Large Scale Integr (VLSI) Syst* 8(3):299–316. doi: [10.1109/92.845896](https://doi.org/10.1109/92.845896)
4. Bergonzini C, Brunelli D, Benini L (2009) Algorithms for harvested energy prediction in batteryless wireless sensor networks. In: 3rd international workshop on advances in sensors and interfaces, 2009. IWASI 2009, pp 144–149 doi: [10.1109/IWASI.2009.5184785](https://doi.org/10.1109/IWASI.2009.5184785)
5. Chen D, Varshney PK (2004) QoS support in wireless sensor networks: a survey. In: Proceedings of international conference on wireless networks Las Vegas, NV
6. Components F (2011) Bp solar—msx-005f—solar panel
7. Cooperation M (2010) Micaz datasheet
8. Escolar S, Chessa S, Carretero J (2012) Optimization of quality of service in wireless sensor networks powered by solar cells. In: 10th IEEE international symposium on parallel and distributed processing with applications, Madrid, Spain, p 8
9. Felemban E, Lee CG, Ekici E (2006) Mmspeed: multipath multi-speed protocol for qos guarantee of reliability and timeliness in wireless sensor networks. *IEEE Trans Mobile Comput* 5(6): 738–754. doi: [10.1109/TMC.2006.79](https://doi.org/10.1109/TMC.2006.79)
10. chun Feng W, Feng X, Ce R (2008) Green supercomputing comes of age. *IT Prof* 10(1):17–23. doi: [10.1109/MITP.2008.8](https://doi.org/10.1109/MITP.2008.8)
11. He T, Stankovic J, Lu C, Abdelzaher T (2003) Speed: a stateless protocol for real-time communication in sensor networks. In:

- Proceedings of 23rd international conference on distributed computing systems, pp 46–55. doi:[10.1109/ICDCS.2003.1203451](https://doi.org/10.1109/ICDCS.2003.1203451)
12. Iyer R, Kleinrock L (2003) Qos control for sensor networks. In: IEEE international conference on communications, 2003. ICC '03, vol 1, pp 517 – 521. doi:[10.1109/ICC.2003.1204230](https://doi.org/10.1109/ICC.2003.1204230)
 13. Kansal A, Hsu J, Zahedi S, Srivastava MB (2007) Power management in energy harvesting sensor networks. ACM Trans Embed Comput Syst 6(4). doi:[10.1145/1274858.1274870](https://doi.org/10.1145/1274858.1274870)
 14. Kopetz H, Grunsteidl G (1993) Ttp—a time-triggered protocol for fault-tolerant real-time systems. In: The twenty-third international symposium on fault-tolerant computing, 1993. FTCS-23. Digest of Papers, pp 524–533. doi:[10.1109/FTCS.1993.627355](https://doi.org/10.1109/FTCS.1993.627355)
 15. Lin C (1998) Qos routing in ad hoc wireless networks. In: Proceedings of 23rd annual conference on local computer networks, 1998. LCN '98, pp 31–40. doi:[10.1109/LCN.1998.727644](https://doi.org/10.1109/LCN.1998.727644)
 16. Meteorology NS, Energy S (2011) Retscreen. <http://eosweb.larc.nasa.gov/sse/RETSscreen/>
 17. Moser C, Chen JJ, Thiele L (2009) Power management in energy harvesting embedded systems with discrete service levels. In: Proceedings of the 14th ACM/IEEE international symposium on low power electronics and design, ISLPED '09, pp 413–418. ACM, New York, NY. doi:[10.1145/1594233.1594338](https://doi.org/10.1145/1594233.1594338)
 18. Murugesan S (2008) Harnessing green it: principles and practices. IT Prof 10(1):24–33. doi:[10.1109/MITP.2008.10](https://doi.org/10.1109/MITP.2008.10)
 19. Raghunathan V, Kansal A, Hsu J, Friedman J, Srivastava M (2005) Design considerations for solar energy harvesting wireless embedded systems. In: Proceedings of 4th international symposium on information processing in sensor networks, IPSN '05. IEEE Press, Piscataway, NJ
 20. Real J, Crespo A (2004) Mode change protocols for real-time systems: a survey and a new proposal. Real-Time Syst 26(2):161–197. doi:[10.1023/B:TIME.0000016129.97430.c6](https://doi.org/10.1023/B:TIME.0000016129.97430.c6)
 21. Salvadori F, de Campos M, Sausen P, de Camargo R, Gehrke C, Rech C, Spohn M, Oliveira A (2009) Monitoring in industrial systems using wireless sensor network with dynamic power management. IEEE Trans Instrum Meas 58(9):3104–3111. doi:[10.1109/TIM.2009.2016882](https://doi.org/10.1109/TIM.2009.2016882)
 22. Sausen PS, de Brito Sousa JR, Spohn MA, Perkusich A, Lima AMN (2008) Dynamic power management with scheduled switching modes. Comput Commun 31:3625–3637. doi:[10.1016/j.comcom.2008.06.019](https://doi.org/10.1016/j.comcom.2008.06.019)
 23. Sha L, Sha L, Rajkumar R, Rajkumar R, Lehoczky J, Lehoczky J, Ramamritham K, Ramamritham K (1988) Mode change protocols for priority-driven preemptive scheduling. Real-Time Syst 1:243–264
 24. Sinha A, Chandrakasan A (2001) Dynamic power management in wireless sensor networks. IEEE Design Test Comput 18(2):62–74. doi:[10.1109/54.914626](https://doi.org/10.1109/54.914626)
 25. Tindell K, Burns A, Wellings (1992) A mode changes in priority preemptively scheduled systems. In: Real-time systems symposium, 1992, pp 100–109. doi:[10.1109/REAL.1992.242672](https://doi.org/10.1109/REAL.1992.242672)
 26. Xia F (2008) Qos challenges and opportunities in wireless sensor/actuator networks. Sensors 8(2):1099–1110. doi:[10.3390/s8021099](https://doi.org/10.3390/s8021099)
 27. Yigitel MA, Incel OD, Ersoy C (2011) Qos-aware mac protocols for wireless sensor networks: a survey. Comput Netw 55:1982–2004. doi:[10.1016/j.comnet.2011.02.007](https://doi.org/10.1016/j.comnet.2011.02.007)
 28. Zhu C, Corson M (2002) Qos routing for mobile ad hoc networks. In: Twenty-first annual joint conference of the IEEE computer and communications societies, INFOCOM 2002, vol 2, pp 958–967. doi:[10.1109/INFCOM.2002.1019343](https://doi.org/10.1109/INFCOM.2002.1019343)